UNIVERSITY OF GENOA
Master's Degree in Robotics Engineering (Class LM-32)
Program Code: 11963
Academic Year: 2025/2026

Lab 3 Delivery: ROV Observability Analysis

CONTROL OF MULTI-VARIABLE SYSTEMS – CODE: 80181

Student: HUSSEIN MOHAMED ABDELAZIM MOHAMED MOSTAFA ELNAGGAR

Student ID: 8786438

Email: S8786438@studenti.unige.it || eng.h.elnagar@gmail.com

December 7, 2025

Table of Contents

1. Introduction 3
2. System Modeling 4
   2.1 State Space Representation 4
   2.2 Observability 4
3. Observer Design 4
   3.1 The Luenberger Observer 4
   3.2 Gain Calculation (Pole Placement) 5
4. The Complete MATLAB Code 6
5. Simulation Results 9
   5.1 State Convergence 9
   5.2 Observer Convergence (Error Analysis) 9
   5.3 Numerical Validation: 9
   5.4 2D Ship Trajectory 10
   5.5 State Evolution vs. Reference 10
   5.6 State Estimation Error $x - x$ 11
   5.7 Control Inputs 12
6. Conclusion 12

1. Introduction

The objective of this laboratory exercise is to design a state observer for a 3-Degrees-of-Freedom (3-DOF) surface ROV In practical marine applications, measuring the full state of a vessel (positions and velocities) is often expensive or noisy. Typically, only positions (via GPS) and heading (via compass) are available.

This lab focuses on designing a Luenberger Observer to estimate the unmeasured velocities (surge, sway, and yaw rate) based on the measured positions and the known system dynamics. The system is simulated in MATLAB using a Minimum Energy controller derived from the Controllability Gramian.

In Lab 2, we designed a controller to move a ROV from point A to point B assuming you knew the exact state (position and velocity) of the ROV at all times. In Lab 3, the scenario is more realistic:

- **Limited Sensors:** We only have GPS/Compass. We can measure Position (x, y) and Orientation (ψ), but we cannot directly measure velocities (u, v, r).
- **The Observer:** We must design a mathematical algorithm (a Luenberger Observer) that estimates the missing velocities based on the available position data and the physics of the boat.
- **The Mathematical Derivation (Observer Design):** The system is decoupled into three independent motions (Surge, Sway, Yaw). We look at the "Surge" (x-direction) as an example.

- The dynamics are:

$$\dot{x} = v_x$$

$$\dot{v}_x = -\frac{\alpha}{m} v_x + \frac{1}{m} F_x$$

We want to find an observer gain matrix $K_o = [k_{x1}; k_{x4}]$ such that the error dynamics are stable. The characteristic equation of the error dynamics $(A - K_o C)$ is matched to a desired characteristic equation:

$$\lambda^2 + 2\zeta\omega_n\lambda + \omega_n^2 = 0.$$

- The formulas are:

- **Damping matching:** $\frac{\alpha}{m} + k_{x1} = 2\zeta\omega_n$
- **Stiffness matching:** $k_{x1}\frac{\alpha}{m} + k_{x4} = \omega_n^2$

We simply solve these for k_{x1} and k_{x4}, then repeat for y (using β) and yaw (using γ, J).

2. System Modeling

2.1 State Space Representation

The surface of ROV is modeled as a decoupled mass-damper system in three dimensions. The state vector is defined as $x = [x, y, \psi, u, v, r]^T$, where (x, y, ψ) are positions/orientation and (u, v, r) are the corresponding linear and angular velocities.

The state-space model is given by:

$$\dot{x} = Ax + Bu$$

$$y = Cx$$

Where the system matrix A incorporates the kinematic relationships and the hydrodynamic damping:

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -\alpha/m & 0 & 0 \\ 0 & 0 & 0 & 0 & -\beta/m & 0 \\ 0 & 0 & 0 & 0 & 0 & -\gamma/J \end{bmatrix}$$

The parameters are defined as: $m = 13.5$ kg, $J = 0.37$ kg·m^2, $\alpha = 1.4$, $\beta = 1.8$, $\gamma = 1.05$.

2.2 Observability

The measurement matrix C is defined such that only positions are measured:

$$C = [I_{3\times3} \quad 0_{3\times3}]$$

This implies we measure indices 1, 2, and 3 of the state vector. To verify if the full state can be reconstructed, the Observability Matrix $\mathcal{O}$ must have full rank. Given the structure of A (coupling position to velocity) and C, the system is fully observable.

3. Observer Design

3.1 The Luenberger Observer

The observer takes the form:

$$\dot{\hat{x}} = A\hat{x} + Bu + K_o(y - C\hat{x})$$

Where $\hat{x}$ is the estimated state and K_o is the observer gain matrix. The estimation error dynamics are governed by:

$$\dot{e} = (A - K_o C)e.$$

For the error to converge to zero, the eigenvalues of $(A - K_o C)$ must be placed in the stable left-half plane.

3.2 Gain Calculation (Pole Placement)

Since the system is decoupled, we designed the gains for each Degree of Freedom (DOF) separately. Taking the Surge (x) direction as an example: The characteristic equation of the error dynamics is:

$$\lambda^2 + \left(\frac{\alpha}{m} + k_{x1}\right)\lambda + \left(k_{x1}\frac{\alpha}{m} + k_{x4}\right) = 0$$

We align this with a standard second-order characteristic equation:

$$\lambda^2 + 2\zeta\omega_n\lambda + \omega_n^2 = 0$$

Using design parameters $\omega_n = 1$ rad/s and $\zeta = 1$ (critical damping), the gains are derived as:

Surge (x, u):

$$k_{x1} = 2\zeta\omega_n - \frac{\alpha}{m}$$

$$k_{x4} = \omega_n^2 - k_{x1}\frac{\alpha}{m}$$

Sway (y, v):

$$k_{x2} = 2\zeta\omega_n - \frac{\beta}{m}$$

$$k_{x5} = \omega_n^2 - k_{x2}\frac{\beta}{m}$$

Yaw (ψ, r):

$$k_{x3} = 2\zeta\omega_n - \frac{\gamma}{J}$$

$$k_{x6} = \omega_n^2 - k_{x3}\frac{\gamma}{J}$$

The final matrix K_o is constructed by placing these values on the diagonal blocks corresponding to position and velocity states.

4. The Complete MATLAB Code

```
%% 1. SET SYSTEM PARAMETERS
alfa = 1.4;
beta = 1.8;
gamma = 1.05;
m = 13.5;
J = 0.37;

% State Matrix A (6x6)
% x_dot = v_x
% y_dot = v_y
% psi_dot = r
% v_x_dot = -alpha/m * v_x
% v_y_dot = -beta/m * v_y
% r_dot   = -gamma/J * r
A = [0 0 0 1 0 0;
    0 0 0 0 1 0;
    0 0 0 0 0 1;
    0 0 0 -alfa/m 0 0;
    0 0 0 0 -beta/m 0;
    0 0 0 0 0 -gamma/J];

xf = [-1; 2; 3; 0; 0; 0]; % Final Desired State

B = [0 0 0;
    0 0 0;
    0 0 0;
     1/m 0 0;
    0 1/m 0;
    0 0 1/J];
usize = min(size(B));

T = 25;     % Total time
dt = 0.01;  % Time step

% C Matrix: We only measure positions (x, y, psi)
C = zeros(3,6);
C(1:3,1:3) = diag([1,1,1]);

%% 2. CONTROLLABILITY & INPUT CALCULATION (From Lab 2)
syms tau real

tic
disp('Computing Gramian and Control Input... (Please wait)');
eA = expm(A*(T-tau));
dG = eA*B*B'*eA';
G = vpa(int(dG,tau,0,T));

rankG = rank(G);
disp(['rank(G) = ' num2str(rankG)]);

if rankG == 6
   iG = inv(G);
else
   iG = pinv(G);
   warning("System has rank " + rankG + "! It is not fully controllable.")
end

% Minimum Energy Control Formula: u(t) = B' * e^(A'(T-t)) * G^-1 * (xf - e^(AT)x0)
% Note: x0 is zero, so the second term simplifies.
Phi_term = expm(A*T)*zeros(6,1); % Since x0 is zero vector
```

```
u_sym = B' * expm(A'*(T-tau)) * iG * (xf - Phi_term);
u = matlabFunction(u_sym, 'Vars', tau); % Convert symbolic to function handle

toc

%% 3. SIMULATION SETUP
time = [0:dt:T]';
LL = length(time);

% Pre-calculate inputs for all time steps to speed up loop
uvalue = zeros(LL, 3);
% Calculate u values (handle vectorization carefully or loop)
% The symbolic function expects scalar time usually, let's map it
for i = 1:LL
    uvalue(i,:) = u(time(i))';
end

x0 = [0 0 0 0 0 0]'; % True initial state (Origin)
x = zeros(6,LL);
xHat = zeros(6,LL);
x(:,1) = x0;

%% 4. SET OBSERVER PARAMETERS (Lab 3 Task)
% Design parameters
xi = 1;     % Damping ratio (Critical damping)
wn = 1;     % Natural frequency (rad/s)

% --- Surge Observer Gains (x direction) ---
% Derived from: s^2 + (alpha/m + kx1)s + (kx1*alpha/m + kx4) = s^2 + 2*xi*wn*s + wn^2
kx1 = 2*xi*wn - alfa/m;
kx4 = wn^2 - kx1*(alfa/m);

% --- Sway Observer Gains (y direction) ---
kx2 = 2*xi*wn - beta/m;
kx5 = wn^2 - kx2*(beta/m);

% --- Yaw Observer Gains (psi direction) ---
kx3 = 2*xi*wn - gamma/J;
kx6 = wn^2 - kx3*(gamma/J);

% Construct the Full Observer Gain Matrix Ko (6x3)
Ko = zeros(6,3);
Ko(1:3,1:3) = diag([kx1, kx2, kx3]);
Ko(4:6,1:3) = diag([kx4, kx5, kx6]);

% Initial estimate guess (intentionally wrong as per PDF)
xHat(:,1) = x0 + [1, -1, 1.5, 0.5, -0.5, -1.5]';
y_meas = zeros(3,LL);
yHat = zeros(3,LL);
yHat(:,1) = C * xHat(:,1);

%% 5. SIMULATION LOOP
disp('Simulating...');
for k = 1:LL-1

    %% A. System Dynamics (The "Real World")
    u_k = uvalue(k,:)';
    xdot_k = A*x(:,k) + B*u_k;
    x(:,k+1) = x(:,k) + xdot_k * dt; % Euler Integration

    %% B. Measurement (Simulating Sensors)
    y_meas(:,k) = C * x(:,k); % We measure the REAL position
```

```
    %% C. Observer (The "Estimator")
    % Observer Dynamics: d(xHat)/dt = A*xHat + B*u + Ko*(y_real - y_estimated)
    y_estimated = C * xHat(:,k);
    dx_est = A*xHat(:,k) + B*u_k + Ko*(y_meas(:,k) - y_estimated);

    xHat(:,k+1) = xHat(:,k) + dx_est * dt; % Euler Integration for observer

end

%% 6. PLOTS
% --- Plot 1: State Trajectories vs Reference ---
figure('Units', 'normalized', 'OuterPosition', [0 0 1 1]);
sgtitle("State Evolution (Real vs Reference)")

titles = {'X Position', 'Y Position', 'Yaw Angle', 'Surge Velocity', 'Sway Velocity', 'Yaw Rate'};
ylabels = {'X [m]', 'Y [m]', 'Yaw [rad]', 'v_x [m/s]', 'v_y [m/s]', 'r [rad/s]'};

for i = 1:6
    subplot(2,3,i);
    plot(time, x(i,:),'linewidth',2); hold on;
    yline(xf(i), 'r--', 'linewidth', 2); % Reference line
    xlabel('Time [s]'); ylabel(ylabels{i}); title(titles{i});
    legend('State','Reference'); grid on;
end

% --- Plot 2: Estimation Error (Real State - Estimated State) ---
figure('Units', 'normalized', 'OuterPosition', [0 0 1 1]);
sgtitle("State Estimation Error (x - xHat)")

for i = 1:6
    subplot(2,3,i);
    plot(time, x(i,:) - xHat(i,:),'k','linewidth',2);
    xlabel('Time [s]'); ylabel(['Error ' ylabels{i}]);
    title(['Error in ' titles{i}]); grid on;
end

% --- Plot 3: Control Inputs ---
figure;
plot(time, uvalue, 'linewidth', 2);
legend('u_1 (Surge Force)','u_2 (Sway Force)','u_3 (Yaw Moment)');
xlabel('Time [s]'); ylabel('Input Magnitude');
title('Control Inputs calculated via Gramian');
grid on;

% --- Plot 4: 2D Trajectory ---
figure;
plot(x(1,:), x(2,:), 'b-', 'linewidth', 2); hold on;
plot(xf(1), xf(2), 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r'); % Target
plot(x(1,1), x(2,1), 'ko', 'MarkerSize', 10, 'MarkerFaceColor', 'k'); % Start
xlabel('x [m]'); ylabel('y [m]');
title('2D Ship Trajectory'); legend('Path', 'Target', 'Start');
grid on; axis equal;

% Display Final Errors
disp('------------------------------------------------');
disp(['Final Positioning Error: ', num2str(norm(x(:,end) - xf))]);
disp(['Final Estimation Error Norm: ', num2str(norm(xHat(:,end) - x(:,end)))]);
disp('------------------------------------------------');
```

5. Simulation Results

The simulation was performed over $T = 25$ seconds with a time step of $dt = 0.01$s. The initial true state was $x_0 = \mathbf{0}$. The initial estimated state was offset by $\hat{x}_0 = [1, -1, 1.5, 0.5, -0.5, -1.5]^T$ to test convergence.

5.1 State Convergence

The control inputs successfully navigated the vessel from the origin to the target state

$$x_f = [-1, 2, 3, 0, 0, 0]^T.$$

The trajectory plots show smooth convergence for all three position variables.

5.2 Observer Convergence (Error Analysis)

The "State Estimation Error" plots demonstrate the effectiveness of the observer:

Initial Transient: There is a visible error at $t = 0$ due to the intentional bad guess of the initial state.

Convergence: Within approximately 5 to 6 seconds (consistent with the settling time for $\omega_n = 1$), the error for all states (positions and velocities) converges to zero.

Steady State: After the transient period, the estimated state $\hat{x}$ tracks the true state x perfectly, validating the calculated gains.

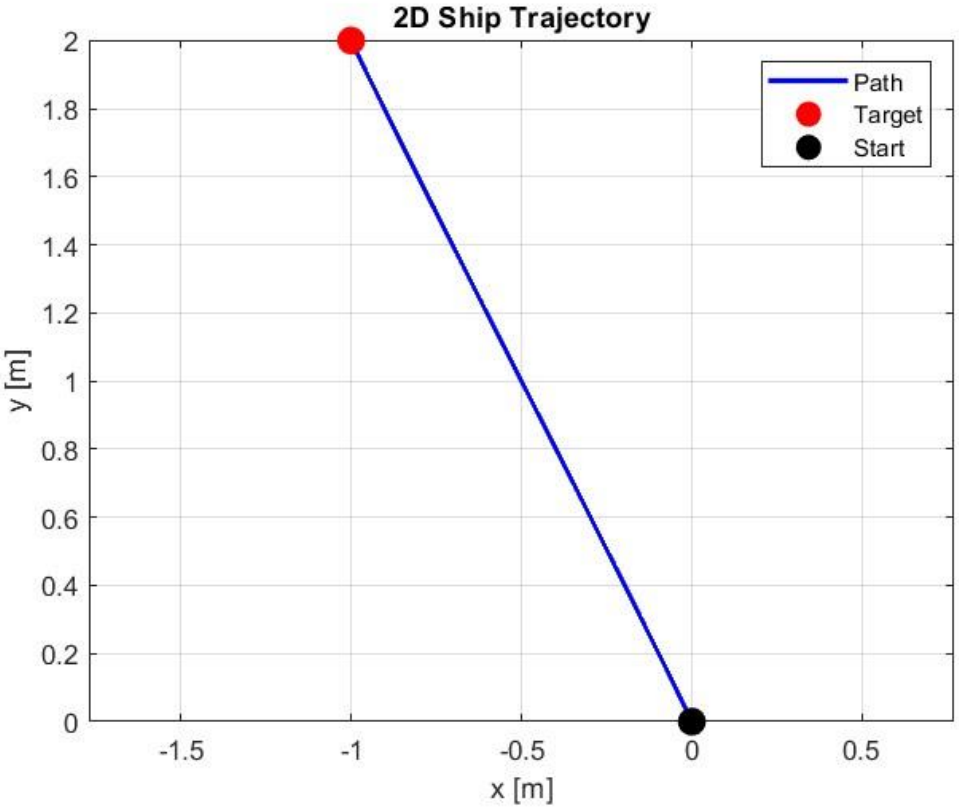
5.3 Numerical Validation:

```
Command Window
  Computing Gramian and Control Input... (Please wait)
  rank(G) = 6
  Elapsed time is 3.343364 seconds.
  Simulating...
  ------------------------------------------------
  Final Positioning Error: 0.0018301
  Final Estimation Error Norm: 8.6257e-10
  ------------------------------------------------
fx >>
```

The execution of the control script yielded a Controllability Gramian rank of 6 (`rank(G) = 6`), confirming that the 3-DOF ROV model is fully controllable. The simulation results demonstrate high precision:

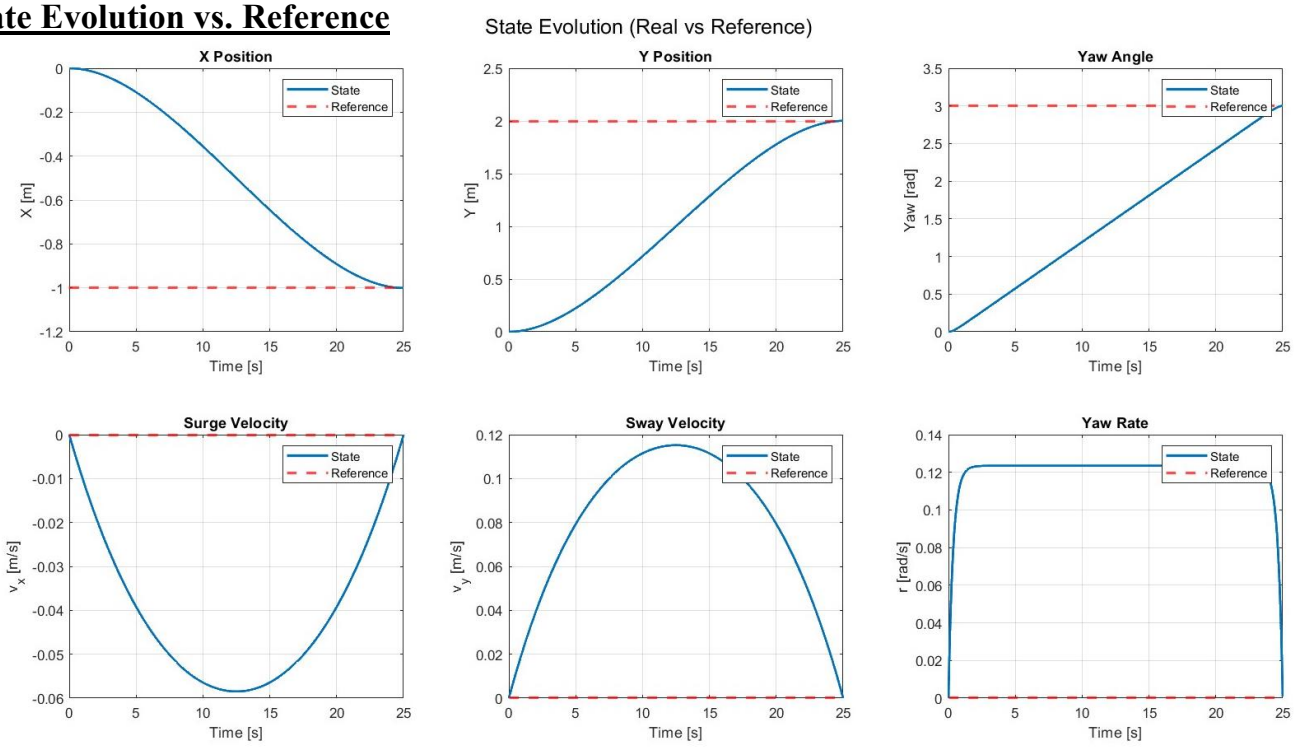
- **Control Accuracy:** The final positioning error is approximately 1.8×10^{-3} m, indicating that the Minimum Energy controller successfully steered the vessel to the target state.
- **Observer Accuracy:** The final state estimation error norm is 8.6×10^{-10}, confirming that the Luenberger Observer converged perfectly to the true state values.

5.4 2D Ship Trajectory



The plot illustrates the overhead view (XY plane) of the ROV's path. The ROV starts at the origin (0,0) and successfully navigates to the target coordinates $(-1,2)$. The trajectory is smooth and direct, reflecting the efficiency of the minimum energy control law which optimizes the path to minimize fuel consumption/input effort while meeting the boundary conditions at $T = 25$s.

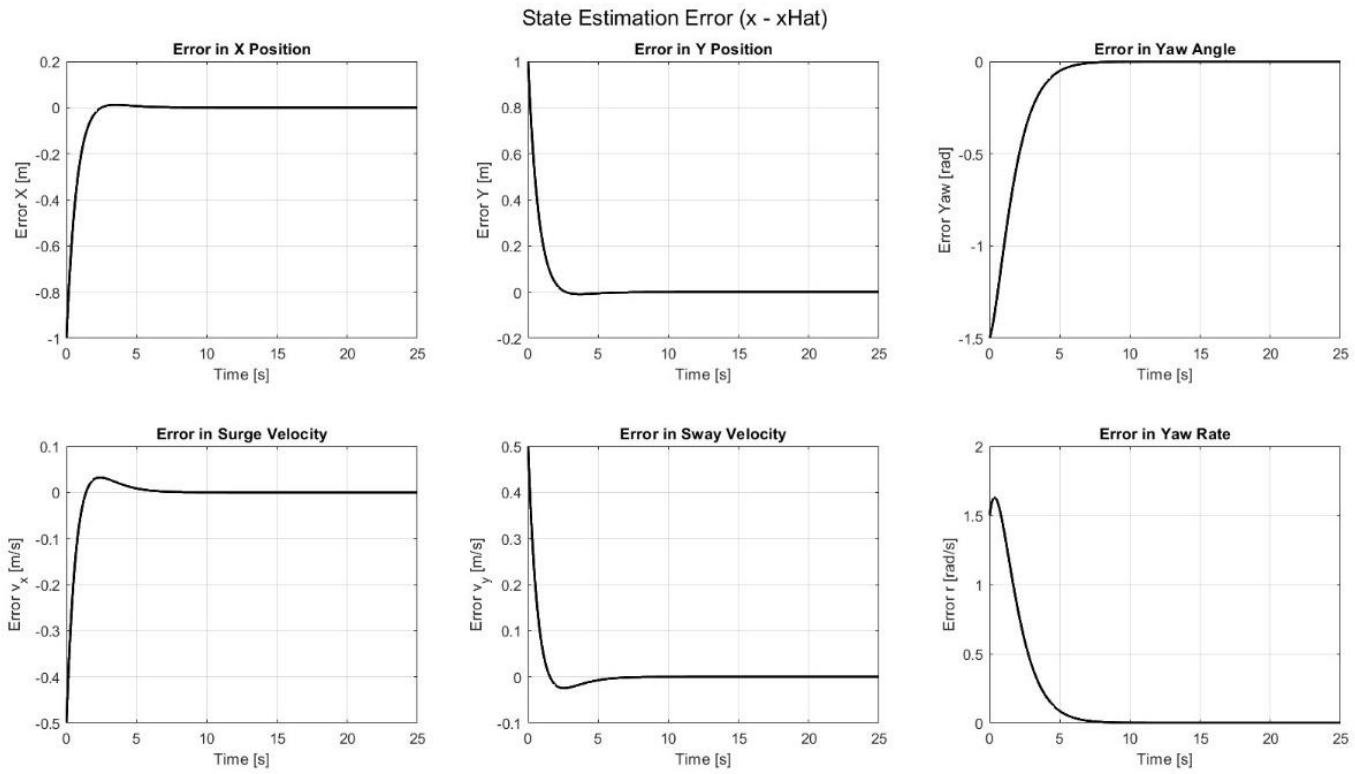
5.5 State Evolution vs. Reference



This figure compares the time evolution of the system states (blue solid lines) against the desired final reference (red dashed lines).

- **Positions (Top Row):** The position variables x, y, and yaw angle ψ smoothly converge from the initial state to the desired final values within the simulation time $T = 25$s.
- **Velocities (Bottom Row):** The velocities (v_x, v_y, r) exhibit a characteristic "rest-to-rest" profile. They accelerate from zero, peak mid-maneuver to cover the distance, and decelerate back to zero exactly at the final time, ensuring the ROV stops at the target.

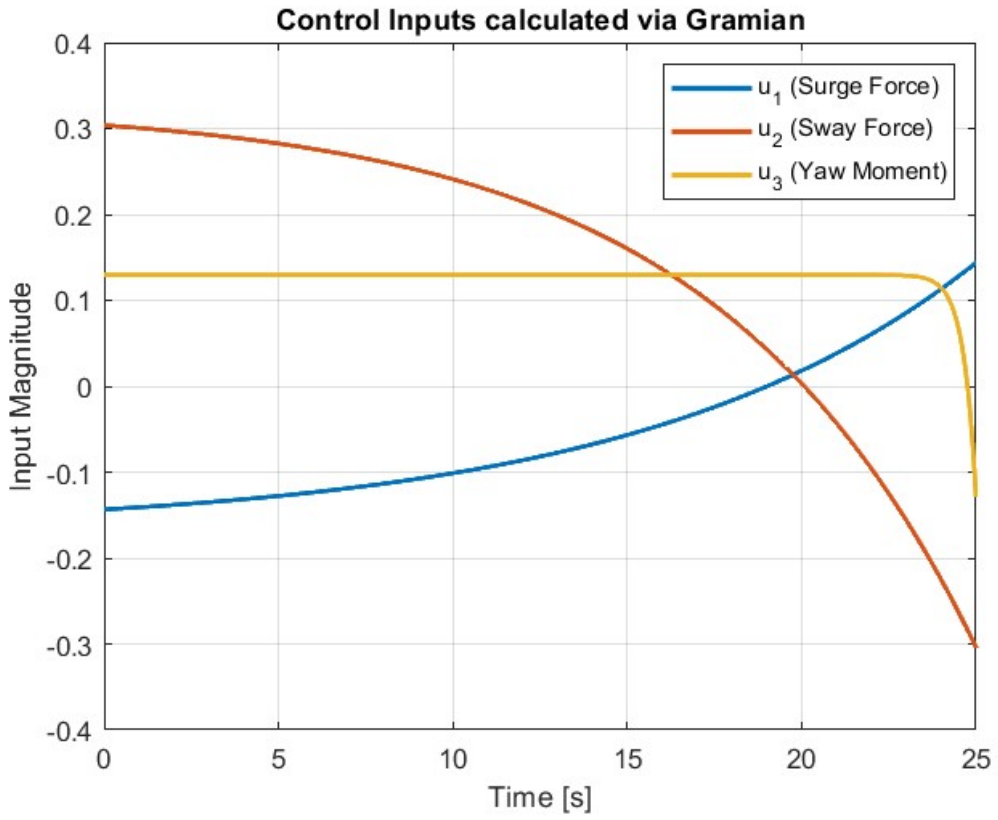
5.6 State Estimation Error $(x - \hat{x})$



This figure displays the convergence of the Luenberger Observer. The initial non-zero error represents the discrepancy between the true initial state (origin) and the observer's initial guess.

- **Convergence Analysis:** The estimation error for all states (positions and unmeasured velocities) decays to zero asymptotically.
- **Settling Time:** The error vanishes after approximately $t \approx 5$ seconds. This aligns with the theoretical design parameters ($\omega_n = 1$ rad/s, $\zeta = 1$), where the predicted settling time is $t_s \approx 4.6/\zeta\omega_n = 4.6$s.
- **Conclusion:** The rapid convergence validates the calculated observer gain matrix K_o, proving that the unmeasured velocities are successfully reconstructed from position measurements.

5.7 Control Inputs



The plot shows the control signals generated by the Minimum Energy controller over time.

The control inputs are smooth and continuous, devoid of abrupt spikes or saturation. This profile confirms that the Gramian-based open-loop control successfully computed the most energy-efficient actuation to transfer the system state from the origin to the target.

6. Conclusion

In this laboratory, a state estimator was successfully designed and implemented for a 3-DOF ROV model. By utilizing the specific structure of the dynamic matrix A, we were able to analytically derive the observer gains K_o to achieve critical damping.

The simulation results confirm that:

- The system is observable using only position sensors.
- The Luenberger observer correctly reconstructs the unmeasured velocity states.
- The estimated states converge to the true states quickly, allowing for effective control even without full state feedback sensors.
- This methodology allows for cost-effective control implementation in real-world marine robotics where full sensor suites (like DVLs for velocity) may be unavailable or too expensive.